AdsKG-Agent — Full Design Document

Companion to the 2-page submitted design doc (v5) · AI Engineering Cohort capstone · Anshika Chaurasia (solo) · October 2026

Section	Section	Section
1. Summary & goals	7. Agent design (single vs. multi-agent)	13. Repository layout
2. Problem & users	8. MCP server	14. Build order & done-criteria
3. Requirements	9. Agent harness	15. Risks
4. Data & warehouse model	10. Evaluation plan & harness	16. Roadmap: What → Why → Next → Act
5. Knowledge graph (Neo4j)	11. User interface	17. Course rubric mapping
6. Vector store & retrieval (Qdrant)	12. Tech stack & justification	18. Glossary

1. Summary & Goals

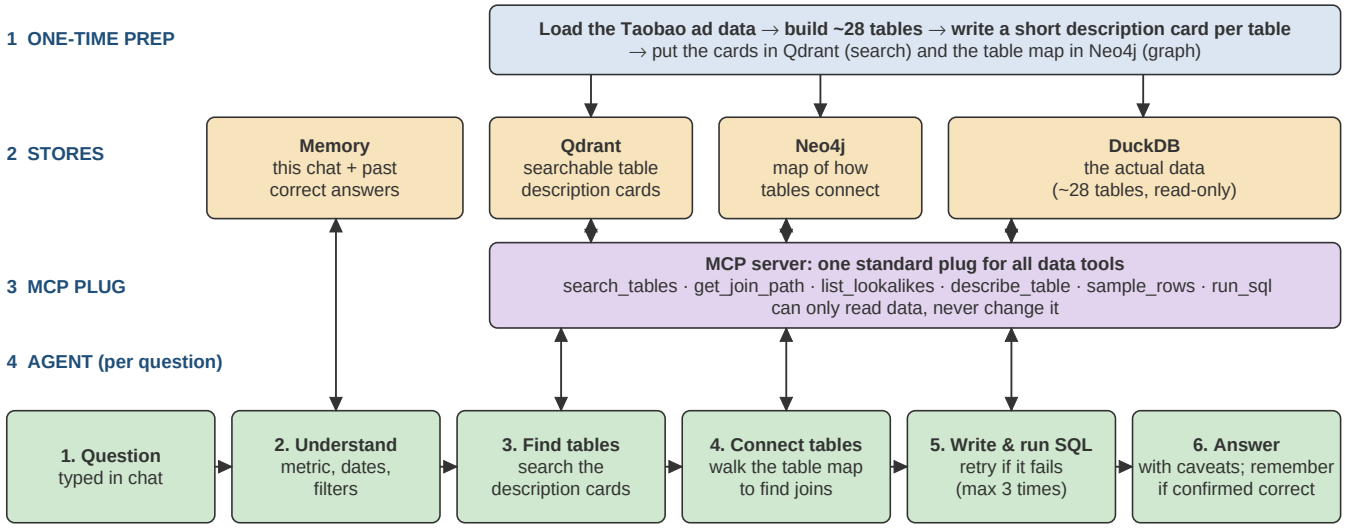
In one sentence: an AI assistant that answers ad-performance questions by first *finding* the right table (search), then *connecting* it to related tables (knowledge graph), then writing and running the database query — and remembering what worked.

Goals: (1) answer read-only "what happened" questions over a ~28-table ad warehouse correctly; (2) show, with a fair experiment, how much each technique adds: search, advanced search, knowledge graph, agent, memory; (3) learn and document vector databases, graph databases, GraphRAG, agents, memory, MCP and harnesses in a way that transfers to my company's ads assistant.

Non-goals (this version): explaining *why* a number changed, recommending or taking actions, writing to the database, dashboards. These are the roadmap stages in Section 16.

Success target (fixed before building)	Value
Best version: correct final answer on the test set	≥ 75%
Gain over basic search (B) on medium + tricky questions	≥ +10 percentage points
Follow-up questions answered correctly with memory on	≥ 80%
Typical (median) answer time / cost per question	≤ 10 s / ≤ $0.02

Architecture at a glance



Agent rules (the "harness"): at most 8 steps · at most 3 retries · cost limit per question · says "not sure" instead of guessing

Figure 1. One-time prep fills the stores; for each question the agent understands it, finds and connects tables through the MCP server, runs the query and answers. Details: Sections 4–9.

2. Problem & Users

Users: ad analysts, campaign managers and advertisers' account teams who ask quick data questions and do not know the warehouse layout.

Why it is hard: the same metric exists in several look-alike tables that differ only by *grain* (hourly / daily / weekly), *traffic quality* (all vs. suspicious clicks removed), *counting* (total views vs. unique people) and *freshness* (live-but-incomplete vs. final). The wrong choice returns a believable but wrong number. Questions also need connecting tables (e.g. brand lives on the ad, not on the view).

Hypothesis: the main failure is telling look-alikes apart and missing connecting tables, not merely the size of the prompt. Version A (no search) tests this at 28 tables; the bigger-catalog test (Section 10) tests size.

Example question	Why it is tricky
"CTR for brand 95471 yesterday"	Brand is on the ad table, so views must be joined to ads; "yesterday" = 12 May 2017

Example question	Why it is tricky
"Clicks for campaign 83237 so far today"	Must use the live partial table, not the final daily table (which stops at yesterday)
"Unique users who saw campaign 83237 on 10 May, suspicious traffic removed"	Needs the reach family and the traffic-quality family together
"Now split that by age group" (follow-up)	Needs short-term memory of the previous question and a join to users

3. Requirements

ID	Functional requirement
F1	Accept a plain-English question and the chat so far; return an answer, the SQL, the tables used and warnings
F2	Find candidate tables by searching table cards (keyword + meaning) and re-ranking
F3	Expand candidates with the knowledge graph: connecting tables, look-alike tables, metric definitions
F4	Generate DuckDB SQL, run it read-only, retry up to 3 times on errors or empty results
F5	Remember the conversation (follow-ups) and reuse verified past answers as examples
F6	Expose all data tools through one MCP server usable by any MCP client
F7	Run any version on any question set from a config file and produce all metrics (test harness)

ID	Non-functional requirement
N1 Safety	Cannot change data: read-only connection, single SELECT, 1,000-row cap, 20 s timeout, audit log
N2 Performance	Median ≤ 10 s and ≤ $0.02 per question; MCP overhead measured
N3 Reproducibility	Fixed "now", frozen table cards and questions, versioned configs, saved traces
N4 Fairness	Every version uses the same tools, cards and model; memory learns only from practice questions
N5 Transparency	Every answer shows its SQL and tables; every run is traced in Langfuse

4. Data & Warehouse Model

4.1 Source and quality

Taobao (Alibaba) display-ad dataset, public on Kaggle: **raw_sample** (26.56M ad views, 1.37M clicks, CTR 5.14%, 1.14M users, 6–13 May 2017, 2 placements), **ad_feature** (847k ads: campaign, advertiser, brand, category, price), **user_profile** (1.06M users: gender, age group, spending level, shopping depth, student flag, city size). Issues fixed on load: reserved column name `user` renamed; literal "NULL" brands converted (29% of ads have no brand); 1.5M views have no user profile, so user joins must be LEFT JOINs.

4.2 Fixed rules (written before any test question)

- **Now** = 2017-05-13 23:59:59 (China time). "Yesterday" = 12 May; "today" = 13 May.
- **Final** daily/weekly tables cover 6–12 May (a simulated 1-day delay). **Live** tables cover only 13 May: the last 6 hours, or today so far.
- **Suspicious traffic:** a user is flagged for a given hour if their clicks in that hour exceed a threshold chosen from the click distribution (e.g. the 99.9th percentile); *valid* tables exclude flagged user-hours.
- **Click value** = clicks × item price, always labelled an estimate (no spend or revenue in the data).

4.3 The ~28 tables

Family	Tables	Grain / note
Staging (3)	stg_impressions, stg_ads, stg_users	cleaned copies of the raw files
Dimensions (7)	dim_adgroup, dim_campaign, dim_advertiser, dim_brand, dim_category, dim_placement, dim_user	one row per entity; labels for coded fields
Event fact (1)	fct_impressions	one row per ad view, with click flag and hour
Time grain (4)	agg_adgroup_daily, agg_campaign_hourly, agg_campaign_daily, agg_campaign_weekly	look-alikes by grain (ROLLS_UP_TO)
Traffic quality (2)	agg_campaign_daily_valid, agg_brand_daily_valid	look-alikes of the all-traffic versions
Reach (2)	agg_campaign_reach_daily, agg_brand_reach_daily	unique users instead of views
Freshness (2)	rt_campaign_last6h, rt_brand_today_partial	live tables for 13 May only
Entity summaries (3)	agg_brand_daily, agg_category_daily, agg_advertiser_daily	daily, all traffic
Audience (2)	agg_campaign_demo_daily, agg_campaign_city_tier_daily	by gender × age group; by city size
Value (1)	agg_campaign_click_value_daily	estimated click value
Placement (1)	agg_placement_hourly	2 placements only

4.4 Table cards

Each table gets a card, generated from DuckDB's information_schema and column statistics, drafted in plain language by an LLM, then checked and frozen by hand. The same cards are used by every version.

```
table: agg_campaign_daily_valid          family: traffic_quality     grain: day   freshness: final (6-12 May)
description: Daily views, clicks and CTR per campaign with suspicious user-hours removed.
columns: date, campaign_id, impressions, clicks, ctr
example values: campaign_id 83237, 391; ctr 0.031, 0.058
joins: campaign_id -> dim_campaign.campaign_id
look-alike of: agg_campaign_daily (differs_by = traffic_filter)
```

4.5 Bigger-catalog distractors

To test scale, cards for public TPC-DS (24 tables, generated in DuckDB) and BIRD schemas are added to the catalog, growing it to ~100 and ~250 tables. They are tagged `is_distractor` and never appear in a correct answer.

5. Knowledge Graph (Neo4j)

Node	Key properties
Table	name, family, grain, freshness, row_count, description, is_distractor
Column	table, name, type, description
Metric	name (CTR, reach, click value), formula
BusinessTerm	text ("views", "impressions", "people reached")
Dimension	name (advertiser, campaign, adgroup, brand, category)
VerifiedQuery	question, sql, verified_by (long-term memory, practice split only during tests)

Relationship	Meaning
(Table)-[:HAS_COLUMN]->(Column)	table structure
(Table)-[:JOINS_TO {left_col, right_col}]->(Table)	how two tables join
(Table)-[:VARIANT_OF {differs_by, value}]->(Table)	look-alike tables (grain, traffic_filter, counting, freshness)
(Table)-[:ROLLS_UP_TO]->(Table)	hourly → daily → weekly
(BusinessTerm)-[:MEANS]->(Metric)-[:COMPUTED_FROM]->(Column)	business words to columns
(Dimension)-[:PARENT_OF]->(Dimension)	advertiser → campaign → adgroup
(VerifiedQuery)-[:USED]->(Table)	memory feeds back into the graph

Map-walking (version D and the get_join_path tool): for every pair of seed tables, find the shortest JOINS_TO route of at most 3 steps; add bridge tables on the route; add VARIANT_OF neighbours with their differs_by values so the LLM can choose; stop at 8 tables in total. Graph is loaded with idempotent MERGE statements, so it can be rebuilt at any time.

```
MATCH (a:Table {name: $t1}), (b:Table {name: $t2})
MATCH p = shortestPath((a)-[:JOINS_TO*..3]-(b))
RETURN [n IN nodes(p) | n.name] AS tables, [r IN relationships(p) | [r.left_col, r.right_col]] AS keys
```

6. Vector Store & Retrieval (Qdrant)

Collection	Contents	Vectors	Payload (filters)
table_cards	one point per table card	dense (meaning) + sparse BM25 (keywords)	table, family, grain, freshness, is_distractor
verified_queries	long-term memory: question → SQL	dense	tables, split, verified_by, date

Retrieval steps by version: B = dense search, top 8 cards. C = (optional) question rewrite into intent → hybrid dense + BM25 search, top 20, fused with reciprocal-rank fusion → cross-encoder re-rank (bge-reranker) → top 8; each of the three pieces can be switched off for the ablation. D = C + map-walking. Filters from the rewritten intent (e.g. grain = hour) are applied as payload filters.

Embeddings: text-embedding-3-small once the course key arrives; a local FastEmbed model is used before that so nothing is blocked. The model is fixed before the final runs and identical for all versions.

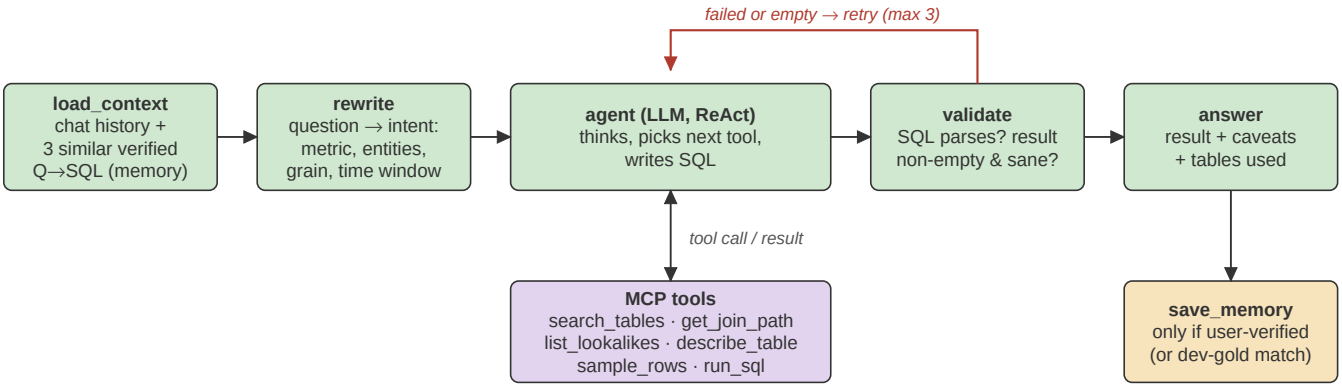
7. Agent Design

7.1 Single agent or multi-agent? Decision: single agent

Option	For	Against	Verdict
Fixed pipeline (no agent)	Cheap, predictable, easy to debug	Cannot recover from a bad search or a failing query	Used for versions B–D
Single ReAct agent with tools	Can re-search, inspect tables and retry; one shared context; simple to trace	More LLM calls than a pipeline	**Chosen for version E**
Multi-agent (planner, retriever, SQL writer, checker)	Separation of roles; good for many different task types	One sequential task here: hand-offs lose context, add cost and delay, and make failures harder to attribute	Rejected now; planned for the roadmap

The "checker" role is kept, but as deterministic code (the validate step), not as a second LLM agent. Multi-agent becomes the right design once the assistant must handle different *kinds* of work (what / why / next / act; Section 16).

7.2 Agent loop (LangGraph)



Harness stops the loop at 8 steps, 3 retries or the cost cap → answers "not sure" instead of guessing

Figure 2. Version E as a LangGraph state graph. Pipelines B–D use the same nodes in a fixed order with no agent node.

State field	Holds
messages	the chat (short-term memory, saved by the checkpointer per thread)
intent	metric, entities, grain, time window, filters (from rewrite)
candidates / join_paths	tables found and how they connect
sql / result / attempts	current query, its result, number of retries
budget	steps, tokens and dollars used so far
caveats / answer / verified	warnings, final text, whether a person confirmed it

7.3 Memory

- **Short-term:** LangGraph checkpointer keyed by conversation thread; long chats are summarised after 6 turns.
- **Long-term:** verified question → SQL pairs in Qdrant (top 3 similar ones are shown to the agent as examples) and user preferences (e.g. "I mean valid traffic by default") in the LangGraph store. Written only when a user clicks "correct" in the UI, or, during testing, when a practice-set answer matches the gold answer. Test questions never enter memory.

7.4 Prompting

System prompt states the fixed "now", the read-only rule, the look-alike families and what each differs_by value means, and asks the agent to state which look-alike it chose and why. Output of rewrite and of the final answer is structured (JSON schema) so the harness can score it.

8. MCP Server ("adskg")

Tool	Input	Output
search_tables	query, k = 8, filters (grain, freshness)	ranked table cards with scores
get_join_path	list of tables, max_hops = 3	bridge tables and join keys
list_lookalikes	table	look-alike tables with differs_by and value
describe_table	table	card, columns and types, row count
sample_rows	table, n = 3	example rows
run_sql	sql	columns, up to 1,000 rows, row count, run time — or a clear error

Also exposes the resource `catalog://tables/{name}`. Built with Python FastMCP; stdio transport for local runs and evaluation, streamable HTTP for the demo. Safety is enforced inside the server: read-only DuckDB connection, sqlglot check that the input is a single SELECT, row cap, timeout, and an audit log of every call. LangGraph connects through langchain-mcp-adapters; Claude Desktop or Cursor can connect unchanged.

9. Agent Harness (rules around the agent)

Control	Setting
Budgets	≤ 8 agent steps, ≤ 3 SQL retries, token and dollar cap per question
Context	≤ 8 table cards (~2k tokens) at a time; tool outputs truncated to 50 rows; summary after 6 turns
Hooks	before a tool: validate SQL; after a tool: truncate output, log to Langfuse
Permissions	only read-only tools exist; enforced in the MCP server
Stop rules	stop when the query succeeds and passes validation, or when a budget runs out → answer "not sure" with what was tried

10. Evaluation Plan & Test Harness

10.1 Benchmark

- 120 questions: 40 easy (one table), 40 medium (joins), 40 tricky (look-alikes). Written from analyst-style templates **before** versions B–E exist; if possible 20 are written by a classmate who has not seen the system.
- Each has gold tables and gold SQL, run against DuckDB and checked by hand. Split: 40 practice, 80 test (test is only used for final runs). Plus 20 follow-up conversations of 3–4 turns (5 practice, 15 test) and paraphrased repeats of practice questions for the memory test. All published in the repo.

10.2 Metrics

Metric	Definition
Execution accuracy	result rows equal the gold rows (order ignored unless the question asks for an order; numbers equal within 1e-6; column names ignored)
Table precision / recall / F1	tables used by the final SQL vs. gold tables (all versions)
Recall@k	share of gold tables among the top-k retrieved (versions B–E)
Join-path correctness	every gold join key pair appears in the SQL
Retry rate	share of questions needing ≥ 1 retry
Latency / tokens / cost	median and 95th percentile per question
Follow-up accuracy	execution accuracy on turns 2+ of conversations, memory on vs. off

10.3 LLM-as-judge

A larger model, different from the one that writes SQL, sees the question, the SQL result and the answer (not the version name) and scores three criteria from 1 to 5: **relevance** (answers what was asked), **caveats** (mentions incomplete live data, estimated value, filters used), **faithfulness** (every number in the text appears in the result). I grade 30 answers myself; agreement is reported as Cohen's kappa.

10.4 Experiments

Experiment	Versions	What changes	Main metric
E1 Main comparison	A, B, C, D, E	technique	execution accuracy, table F1
E2 Smart-search ablation	C	hybrid / re-rank / rewrite on or off	recall@k, execution accuracy
E3 Graph settings (practice set)	D	max hops 1–3, table cap 4/8/12	table F1
E4 Memory	E	memory on vs. off	follow-up accuracy
E5 Bigger catalog	A, B, C, D	28 / ~100 / ~250 tables	execution accuracy, tokens
E6 Tools (stretch)	E vs. E′	custom vs. generic MCP servers	execution accuracy, steps

Statistics: each configuration runs 3 times; results are reported with 95% bootstrap confidence intervals (1,000 resamples) and each step A→B→C→D→E is checked with a paired McNemar test on the 80 test questions. **Final choice:** highest test execution accuracy; when confidence intervals overlap, the cheaper and faster version wins.

10.5 Test harness

```
# configs/D_test.yaml                          python -m harness.run configs/D_test.yaml
version: D            # A | B | C | D | E | E_prime
split: test           # practice | test
catalog: base         # base (28) | plus100 | plus250
runs: 3
retrieval: {k: 8, hybrid: true, rerank: true, rewrite: true}
graph: {max_hops: 3, table_cap: 8}
memory: false
model: {generator: <small chat model>, judge: <larger model>}
```

Outputs per run: `results/<run_id>/answers.jsonl` (question, SQL, tables, result, timing, cost), `metrics.json`, and a Langfuse trace per question. `harness/report.py` builds every table and chart in the final documentation from these files, so all numbers can be regenerated with one command.

10.6 Failure analysis

At least 30 failures are labelled: wrong look-alike, missing connecting table, wrong join key, too many tables, made-up column, wrong time window, SQL error not recovered. Counts per version go into the README together with every approach tried and every pivot made.

11. User Interface (optional)

Streamlit chat. Beside each answer: the table cards retrieved, the graph route used, the SQL, the result table and the warnings, plus "correct / wrong" buttons that feed long-term memory. A sidebar switches between versions A–E for the demo video.

12. Tech Stack & Justification

Tool	Role	Why (alternative considered)
DuckDB	warehouse	fast on a laptop with 26M rows, no server (Postgres)
Qdrant	vector database	dense + sparse hybrid search and payload filters built in (Chroma, pgvector)
Neo4j	graph database	Cypher route queries, persistence, industry standard; NetworkX would suffice at 28 tables but not for the 250-table test or for transfer to work (NetworkX)
LangGraph	agent + memory	explicit state graph, retries, checkpointer and store (Pydantic AI, smolagents)
FastMCP	tool server	one tool layer for every version and any client; safety in one place (plain functions)
Langfuse	tracing	open source, per-step traces for failure analysis (LangSmith)
sqlglot	SQL safety	real parser instead of regex
OpenAI models, bge-reranker, FastEmbed	LLM, re-ranking, local embeddings	course key; open-source re-ranker; local embeddings until the key arrives
Streamlit	UI	fastest way to a usable chat with side panels (Gradio)

13. Repository Layout

```
adskg-agent/
  data/            raw/ (Kaggle CSVs, not committed)   ads.duckdb
  warehouse/       build_tables.sql   rules.md (fixed rules from 4.2)
  catalog/         make_cards.py   cards/*.yaml (frozen)   load_qdrant.py   load_neo4j.py
  mcp_server/      server.py (FastMCP tools + safety)
  agent/           pipelines.py (A-D)   graph.py (E, LangGraph)   memory.py   prompts/
  harness/         run.py   metrics.py   judge.py   report.py
  configs/         *.yaml (one per experiment)
  benchmark/       questions.jsonl   conversations.jsonl   gold/*.sql
  results/         <run_id>/answers.jsonl, metrics.json
  app/             streamlit_app.py
  docs/            design_doc.pdf   README.md (results + failure analysis)
```

14. Build Order & Done-Criteria

Each step must work before the next starts, so a usable, gradeable system exists at every point.

#	Milestone	Done when
M1	Load data, build ~28 tables, write fixed rules	all tables build from one SQL script; row counts checked
M2	Table cards + Qdrant (local embeddings)	search_tables returns sensible cards for 10 sample questions
M3	Benchmark: 120 questions + gold SQL	every gold SQL runs; practice/test split frozen
M4	Versions A and B + test harness	harness prints execution accuracy for A and B on practice
M5	Neo4j graph + version D	get_join_path finds the right bridge tables on 10 checks
M6	MCP server with safety	all tools callable from LangGraph and from Claude Desktop; write attempts blocked
M7	Version E + memory + agent harness	follow-up conversations work; budgets stop runaway loops
M8	Version C + ablations; bigger-catalog test	E2 and E5 results saved
M9	Final runs, LLM judge, statistics	all E1–E5 tables and charts generated by report.py
M10	Stretch E′, UI, README, 3-minute video	submission package complete

15. Risks

Risk	Mitigation
Course API key arrives late	local FastEmbed embeddings; M1–M3 and M5–M6 need no LLM
Version A wins at 28 tables	reported as a real finding; E5 shows where search becomes necessary
Benchmark bias (I wrote it)	rules and questions frozen first; classmate subset; per-tier results; published
LLM-drafted cards leak hints	hand-checked, frozen, identical for all versions
Neo4j / LangGraph / MCP setup trouble	Docker images pinned; each tool tested alone before integration
Too much scope for the time	build order M1→M10; stretch items documented honestly if unfinished
Simulated "now" and estimated value	stated clearly; no question claims real spend or revenue

16. Roadmap: What → Why → What next → Act

Stage	Question	What gets added	Agent design
1. What (this capstone)	"What was CTR yesterday?"	table finding, SQL, memory	single agent
2. Why	"Why did CTR drop 18%?"	metric tree in the graph (value = views × CTR × price); breakdown tools by campaign, placement, audience; contribution ranking	Why agent added
3. What next	"What should I check or change?"	playbooks of causes and fixes searched with RAG; ranked suggestions with evidence	What-next agent added
4. Act	"Move budget from A to B"	optimiser proposes moves; new write tools in MCP; mandatory human approval; audit log	Act agent + approval gate

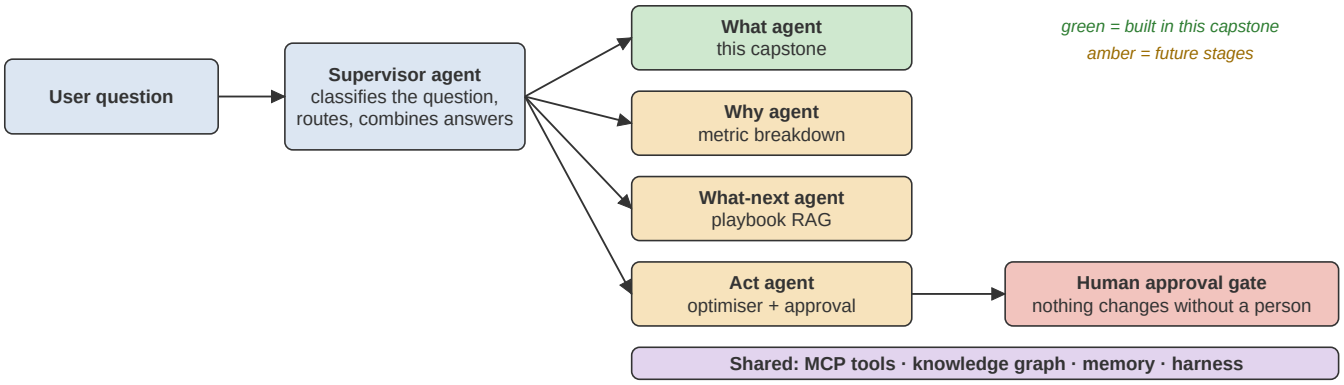


Figure 3. From stage 2 the system becomes multi-agent: a supervisor routes each question to a specialist agent; all specialists share the MCP tools, knowledge graph, memory and harness built in this capstone.

17. Course Rubric Mapping

Course requirement / rubric item	Where it is addressed
Problem specificity	Sections 1–3
Data surface + data processing (ingestion)	Sections 4, 8
Retrieval design	Sections 5, 6
Orchestration, tool use, context handling	Sections 7, 9
Comparative evaluation of ≥ 2 approaches with justified choice	Section 10.4 (E1), final-choice rule
LLM-as-judge + quantitative metrics	Sections 10.2, 10.3
Failure analysis (attempts, failures, pivots)	Section 10.6 and the README
Decision reasoning (frameworks)	Sections 7.1, 12
User interface (optional)	Section 11

18. Glossary

Term	Meaning
RAG	search for relevant information first, then give it to the AI to answer
Vector database	finds text by meaning, not only exact words (Qdrant)
Hybrid search / re-ranking	combine keyword and meaning search; then a second model re-orders the best results
Knowledge graph / GraphRAG	a map of things and how they connect; GraphRAG uses that map to add related context
Agent / ReAct	an AI that alternates between thinking and using tools until the task is done
Single vs. multi-agent	one agent doing the whole task vs. several specialised agents handing work to each other
MCP	Model Context Protocol: a standard way to plug tools into any AI app, like USB for AI tools
Agent harness / test harness	rules around the agent (limits, retries) / one script that runs and scores every version
LLM-as-judge	using a second AI to grade answers, checked against human grades